Ministry of Higher Education and Scientific Research
M'hamed Bougara University of Boumerdès
Faculty of Sciences – Department of Computer Science

Practical Work 1	Module: Advanced Databases	Academic Year: S1 – Master GL / I2A	ISHAK BOUSHAKI

TP 1 — PostgreSQL: Creating, Populating and Modifying the university Database

Learning Objectives: By the end of this TP, students will be able to install PostgreSQL with pgAdmin 4, create the university database and its tables with all attribute-level constraints, represent an N:N relationship using a junction table without modifying the participating tables, and manage constraints (ALTER TABLE, add/modify/drop, ON DELETE SET NULL, ON UPDATE CASCADE). They will also master insertion order and the core SQL commands (INSERT, UPDATE, SELECT with WHERE, AND, OR, BETWEEN, LIKE, ILIKE, IS NULL, ORDER BY) while verifying data integrity.

PART A — PostgreSQL Installation and pgAdmin 4

A.1. Working Environment

In this TP:

- **SQL** is the language used to communicate with the database.
- **PostgreSQL** is the DBMS that executes SQL statements.
- **pgAdmin 4** is the graphical interface used to administer PostgreSQL and execute SQL commands.

SQL = language
PostgreSQL = DBMS
pgAdmin 4 = graphical administration interface

A.2. PostgreSQL Installation Steps

A.2.1. Download the Installer

1. Open the official PostgreSQL download page: https://www.postgresql.org/download/windows/
2. Click **"Download the installer"**.
3. Choose the appropriate Windows version (32-bit or 64-bit).
4. The installer is provided by EDB (EnterpriseDB) and includes:
    - PostgreSQL Server (database server)
    - pgAdmin 4 (graphical administration tool)
    - Stack Builder (additional tools manager)
    - Command Line Tools

A.2.2. Run the Installation Wizard

1. Double-click the downloaded .exe file.
2. Follow the wizard steps:

Step	Action
Welcome	Click "Next"

Step	Action
Installation Directory	Use default C:\Program Files\PostgreSQL\<version>\, click "Next"
Component Selection	Ensure PostgreSQL Server and pgAdmin 4 are checked, click "Next"
Data Directory	Use default path, click "Next"
Superuser Password	Enter password for postgres user, remember this password, click "Next"
Port	Default 5432, keep default, click "Next"
Locale	Select "Default locale" or your preferred locale, click "Next"
Pre-Installation Summary	Confirm settings, click "Next" to start installation

3. Installation takes about 2–5 minutes. Click "Finish" when done.

A.2.3. Verify Installation

1. Open **pgAdmin 4** from the Start menu.
2. On first launch, set a **master password** (can be the same as the postgres user password).
3. Expand the **Servers** node in the left panel.
4. Enter the postgres user password set during installation.
5. Once connected, right-click **Databases → Create → Database** to test database creation.

A.2.4. Using Query Tool

1. In pgAdmin, right-click the target database.
2. Select **Query Tool**.
3. Type SQL statements in the editor on the right.
4. Click the ▶ button or press **F5** to execute.

A.3. Connect to PostgreSQL

1. Open **pgAdmin 4**.
2. Expand **Servers**.
3. Select your PostgreSQL server.
4. Enter the password defined during installation.
5. Verify that the server is connected.

Open **Query Tool** from the server or from an existing database.

PART B — Creating the university Database

B.1. Create the Database

The database used throughout this course is called: university

Step 1 — Open Query Tool

Open Query Tool from the PostgreSQL server.

Step 2 — Create the database

Execute:

CREATE DATABASE university;

Step 3 — Refresh the database list

In pgAdmin:

1. Expand **Databases**.
2. Right-click **Databases**.
3. Select **Refresh**.
4. Verify that university appears.

Step 4 — Connect to university

Open a new Query Tool connected specifically to: university

Important: All tables and data in this TP must be created inside the university database.

Step 5 — Verify the current database

Execute:

SELECT current_database();

Expected result:

university

PART C — Understanding the Initial Relational Schema

C.1. Initial Tables

The database contains four tables:

department(department_id PK, department_name)

class(class_id PK, class_name, department_id FK)

teacher(teacher_id PK, last_name, first_name, salary, department_id FK)

student(student_id PK, last_name, first_name, age, class_id FK)

PK = Primary Key et FK = Foreign Key.

PART D — Creating the Initial Tables

D.1. Create department with All Constraints

```
CREATE TABLE department (
   department_id   INTEGER       PRIMARY KEY,
   department_name VARCHAR(50)   NOT NULL UNIQUE);
```

Constraints defined on attributes:

Attribute	Constraint	Role
department_id	PRIMARY KEY	Uniquely identifies each department; implicitly NOT NULL and UNIQUE
department_name	NOT NULL	Prevents missing department names
department_name	UNIQUE	Prevents duplicate department names

D.2. Create teacher

```
CREATE TABLE teacher (
   teacher_id     INTEGER       PRIMARY KEY,
   last_name      VARCHAR(50)   NOT NULL,
   first_name     VARCHAR(50)   NOT NULL,
   salary         NUMERIC(10,2) DEFAULT 30000,
   department_id  INTEGER       REFERENCES department(department_id)
                        ON DELETE SET NULL
                        ON UPDATE CASCADE);
```

Constraints defined on attributes:

Attribute	Constraint	Role
teacher_id	PRIMARY KEY	Uniquely identifies each teacher
salary	DEFAULT 30000	Default salary when not provided
department_id	ON DELETE SET NULL	If department is deleted, teacher's department_id becomes NULL
department_id	ON UPDATE CASCADE	If department ID changes, teacher's reference updates automatically

D.3. Create class

```
CREATE TABLE class (
   class_id       INTEGER       PRIMARY KEY,
   class_name     VARCHAR(50)   NOT NULL,
   department_id  INTEGER       REFERENCES department(department_id)
                        ON DELETE CASCADE
                        ON UPDATE CASCADE);
```

D.4. Create student

```
CREATE TABLE student (
   student_id   INTEGER       PRIMARY KEY,
```

```
    last_name    VARCHAR(50)   NOT NULL,
    first_name   VARCHAR(50)   NOT NULL,
    age          INTEGER       NOT NULL
                               CHECK (age > 0 AND age < 150),
    class_id     INTEGER       REFERENCES class(class_id)
                               ON DELETE SET NULL
                               ON UPDATE CASCADE);
```

D.5. Verify the Created Tables

In pgAdmin:

Schemas → public → Tables

You should see: department, teacher, class and student

Verify constraints: **Schemas → public → Tables → table name → Constraints**

PART E — The Many-to-Many (N:N) Relationship

E.1. Question: How to Represent "One Teacher Teaches Many Classes and One Class Is Taught by Many Teachers"?

Problem statement: Suppose that **each teacher teaches several classes** and **each class is taught by several teachers**.

Questions:

Q1. Can we represent this relationship by adding a class_id column to the teacher table? Why or why not?

Q2. Can we represent this relationship by adding a teacher_id column to the class table? Why or why not?

Q3. What happens if we try to store multiple values in a single column (e.g., class_id = '10, 20, 30')? What problems does this cause?

Q4. What is the standard relational solution to represent a many-to-many relationship?

Q5. What is this new table called? (junction table, association table, bridge table)

Q6. What columns must this new table contain?

Q7. Why must the combination of these columns be unique?

E.2. Answer: The Junction Table

A many-to-many relationship **cannot** be represented by adding a foreign key to either participating table, because:

- Adding class_id to teacher would allow only **one class per teacher**.
- Adding teacher_id to class would allow only **one teacher per class**.
- Storing multiple values in one column violates **first normal form (1NF)** and makes queries and constraints impossible.

The standard solution is to create a **third table**, called a **junction table** (or association table, or bridge table), that contains:

- the primary key of the first table (teacher_id);

- the primary key of the second table (class_id).

The combination (teacher_id, class_id) forms a **composite primary key**, which guarantees that the same pair cannot be inserted twice.

Key principle: The two participating tables (teacher and class) are **not modified**. The N:N relationship is expressed **entirely** by the new junction table.

E.3. Create teacher_class — The N:N Junction Table

```
CREATE TABLE teacher_class (
   teacher_id   INTEGER   NOT NULL,
   class_id     INTEGER   NOT NULL,
   PRIMARY KEY (teacher_id, class_id),
   FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id)
                ON DELETE CASCADE
                ON UPDATE CASCADE,
   FOREIGN KEY (class_id)   REFERENCES class(class_id)
                ON DELETE CASCADE
                ON UPDATE CASCADE);
```

Explanation of the junction table:

Element	Role
PRIMARY KEY (teacher_id, class_id)	**Composite primary key** — prevents duplicate pairs (same teacher + same class)
FOREIGN KEY (teacher_id)	Ensures the teacher exists
FOREIGN KEY (class_id)	Ensures the class exists
ON DELETE CASCADE	If a teacher or class is deleted, the corresponding rows in teacher_class are automatically deleted
ON UPDATE CASCADE	If a teacher or class ID changes, the reference updates automatically

Why this design is correct:

1. **Does not modify** teacher or class.
2. Contains **one row per pair** (teacher, class).
3. Uses a **composite primary key** to prevent duplicates.
4. Uses **foreign keys** to maintain referential integrity.
5. Can be **extended** with additional attributes (e.g., academic year, semester) without touching the participating tables.

E.4. Updated Relational Schema

The database now contains **five tables**:

PART F — How to Add, Modify, and Drop a Constraint

F.1. Add a Constraint

General syntax:

sql

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name
constraint_type (column_name);
```

Example — Add a CHECK constraint on teacher.salary:

sql

```
ALTER TABLE teacher
ADD CONSTRAINT chk_teacher_salary
CHECK (salary IS NULL OR salary > 0);
```

F.2. Modify a Constraint

PostgreSQL does not allow direct modification of a constraint. You must drop the old constraint and add a new one.

Example — Modify chk_teacher_salary to require salary >= 30000 when not NULL:

sql

```
-- Step 1: Drop the old constraint
ALTER TABLE teacher
DROP CONSTRAINT chk_teacher_salary;
-- Step 2: Add the new constraint
ALTER TABLE teacher
ADD CONSTRAINT chk_teacher_salary
CHECK (salary IS NULL OR salary >= 30000);
```

F.3. Drop a Constraint

General syntax:

```
ALTER TABLE table_name
DROP CONSTRAINT constraint_name;
```

Example — Drop the CHECK constraint on teacher.salary:

```
ALTER TABLE teacher
DROP CONSTRAINT chk_teacher_salary;
```

PART H — Understanding ON DELETE SET NULL and ON UPDATE CASCADE

H.1. Explanation of ON DELETE SET NULL

When a **parent row is deleted**, the foreign key in the **child row** is set to NULL.

Example in the university **database:**

department_id INTEGER REFERENCES department(department_id)

ON DELETE SET NULL

Meaning: If a department is deleted from the department table, the department_id of all teachers belonging to that department becomes NULL. The teachers themselves are **not deleted**.

Why use ON DELETE SET NULL**?**

- To **preserve** the child rows even when the parent disappears.
- To avoid losing important data (e.g., teachers should not be deleted just because their department is removed).
- To allow the child row to be reassigned later.

Example scenario:

DELETE FROM department WHERE department_id = 3;

-- Teachers of department 3 now have department_id = NULL

-- They still exist in the teacher table

Important: ON DELETE SET NULL cannot be used if the foreign key column is declared NOT NULL, because the action would violate the NOT NULL constraint.

H.2. Explanation of ON UPDATE CASCADE

When a **parent primary key is updated**, the foreign key in the **child row** is automatically updated to the new value.

Example in the university **database:**

department_id INTEGER REFERENCES department(department_id)

ON UPDATE CASCADE

Meaning: If a department's department_id is changed in the department table, the department_id of all teachers belonging to that department is **automatically updated** to the new value.

Why use ON UPDATE CASCADE**?**

- To maintain **referential integrity** automatically.
- To avoid orphan rows when primary keys change.
- To simplify maintenance when identifiers need to be renumbered.

Example scenario:

sql

UPDATE department SET department_id = 100 WHERE department_id = 1;

-- All teachers with department_id = 1 now have department_id = 100

H.3. Comparison of Actions

Action	Effect on child rows
ON DELETE CASCADE	Child rows are **deleted** when the parent is deleted
ON DELETE SET NULL	Child foreign key is **set to NULL** when the parent is deleted
ON DELETE RESTRICT	Deletion is **refused** if child rows exist
ON DELETE NO ACTION	Same as RESTRICT (default)
ON UPDATE CASCADE	Child foreign key is **updated** when the parent key changes
ON UPDATE SET NULL	Child foreign key is **set to NULL** when the parent key changes
ON UPDATE RESTRICT	Update is **refused** if child rows exist

H.4. Questions on ON DELETE SET NULL and ON UPDATE CASCADE

Q1. What happens to a teacher's department_id if the referenced department is deleted?

Q2. What happens to a teacher's department_id if the referenced department's department_id is updated?

Q3. Why is ON DELETE SET NULL used for teacher.department_id instead of ON DELETE CASCADE?

Q4. What would happen if ON DELETE CASCADE were used instead for teacher.department_id?

Q5. What would happen if ON UPDATE CASCADE were replaced by ON UPDATE RESTRICT?

Q6. Can ON DELETE SET NULL be used if the foreign key column is declared NOT NULL? Justify your answer.

Q7. Give a concrete SQL example that triggers ON UPDATE CASCADE on the department table.

Q8. In teacher_class, why is ON DELETE CASCADE used instead of ON DELETE SET NULL?

PART I — Insertion Order and Why It Matters

I.1. Why Respect an Insertion Order?

Because of **foreign-key constraints**, a row that references another row **cannot be inserted before** the referenced row exists.

Example: teacher.department_id ———► department.department_id

If you try to insert a teacher with department_id = 1 before department 1 exists, PostgreSQL will reject the insertion with a foreign-key violation.

Therefore, the insertion order must follow the dependency graph:

1. department (no dependencies)
2. teacher (depends on department)
3. class (depends on department)
4. student (depends on class)
5. teacher_class (depends on teacher AND class)

Rules:

- A table can be inserted only after **all tables it references** have been populated.
- The junction table teacher_class must be inserted **last**, because it depends on both teacher and class.

I.2. Questions on Insertion Order

Q1. Why must department be inserted before teacher?

Q2. Why must department be inserted before class?

Q3. Why must class be inserted before student?

Q4. Why must teacher and class be inserted before teacher_class?

Q5. What error would PostgreSQL return if you tried to insert a student with class_id = 99 when class 99 does not exist?

Q6. Can you insert a teacher with department_id = NULL? Why or why not?

Q7. Can you insert a student with class_id = NULL? Why or why not?

Q8. What would happen if you tried to insert a row into teacher_class with a teacher_id that does not exist in teacher?

PART J — Inserting Data

Because of the foreign-key dependencies, insert data in this order:

1. department
2. teacher and class
3. student
4. teacher_class

J.1. Insert Departments

```
INSERT INTO department (department_id, department_name)
VALUES
(1, 'Informatique'),
(2, 'Mathématiques'),
(3, 'Électronique'),
(4, 'Physique'),
(5, 'Chimie');
```

J.2. Insert Teachers

```
INSERT INTO teacher
   (teacher_id, last_name, first_name, salary, department_id)
VALUES
(101, 'Benali', 'Ahmed', 75000, 1),
(102, 'Saidi', 'Nadia', 62000, 1),
(103, 'Karim', 'Samir', 48000, 2),
(104, 'Haddad', 'Amel', 85000, 2),
(105, 'Mansouri', 'Ali', 55000, 3),
```

(106, 'Rahmani', 'Sara', 72000, 3),

(107, 'Bensaid', 'Omar', 45000, 4),

(108, 'Ahmed', 'Yasmine', 68000, 1),

(109, 'Khelifi', 'Nora', NULL, 1);

J.3. Insert Classes

INSERT INTO class

(class_id, class_name, department_id)

VALUES

(10, 'M1-GL', 1),

(20, 'M1-IA', 1),

(30, 'M1-Math', 2),

(40, 'M1-Elec', 3),

(50, 'M1-Physique', 4);

J.4. Insert Students

INSERT INTO student

(student_id, last_name, first_name, age, class_id)

VALUES

(1, 'Ahmed', 'Karim', 22, 10),

(2, 'Ali', 'Sara', 23, 10),

(3, 'Amine', 'Yacine', 21, 20),

(4, 'Benali', 'Lina', 24, 20),

(5, 'Brahimi', 'Omar', 22, 30),

(6, 'Chibani', 'Amel', 23, 30),

(7, 'Dali', 'Sami', 21, 40),

(8, 'Farah', 'Ines', 22, NULL),

(9, 'Ahmed', 'Salim', 25, 10),

(10, 'Ali', 'Nadia', 20, 20);

J.5. Insert Teacher-Class Assignments (N:N Data)

Now populate the junction table. Each row means: **"this teacher teaches this class"**.

INSERT INTO teacher_class (teacher_id, class_id)

VALUES

-- Teacher 101 (Benali Ahmed) teaches M1-GL and M1-IA

(101, 10),

(101, 20),

-- Teacher 102 (Saidi Nadia) teaches M1-GL and M1-Math

(102, 10),

(102, 30),

-- Teacher 103 (Karim Samir) teaches M1-Math

(103, 30),

-- Teacher 104 (Haddad Amel) teaches M1-Math and M1-Elec

(104, 30),

(104, 40),

-- Teacher 105 (Mansouri Ali) teaches M1-Elec

(105, 40),

-- Teacher 106 (Rahmani Sara) teaches M1-Elec and M1-Physique

(106, 40),

(106, 50),

-- Teacher 107 (Bensaid Omar) teaches M1-Physique

(107, 50),

-- Teacher 108 (Ahmed Yasmine) teaches M1-GL and M1-IA

(108, 10),

(108, 20),

-- Teacher 109 (Khelifi Nora) teaches M1-IA

(109, 20);

PART K — Verifying the Inserted Data

K.1. Display All Departments

SELECT * FROM department;

K.2. Display All Teachers

SELECT * FROM teacher;

K.3. Display All Classes

SELECT * FROM class;

K.4. Display All Students

SELECT * FROM student;

K.5. Display All Teacher-Class Assignments

SELECT * FROM teacher_class

```
ORDER BY teacher_id, class_id;
```

PART L — Querying the N:N Relationship

L.1. Display Each Teacher with the Classes They Teach

```
sql
SELECT
    t.teacher_id,
    t.last_name,
    t.first_name,
    c.class_id,
    c.class_name
FROM teacher t
JOIN teacher_class tc ON t.teacher_id = tc.teacher_id
JOIN class c ON tc.class_id = c.class_id
ORDER BY t.teacher_id, c.class_id;
```

L.2. Display Each Class with Its Teachers

```
SELECT
    c.class_id,
    c.class_name,
    t.teacher_id,
    t.last_name,
    t.first_name
FROM class c
JOIN teacher_class tc ON c.class_id = tc.class_id
JOIN teacher t ON tc.teacher_id = t.teacher_id
ORDER BY c.class_id, t.teacher_id;
```

L.3. Count How Many Classes Each Teacher Teaches

```
sql
SELECT
    t.teacher_id,
    t.last_name,
    t.first_name,
    COUNT(tc.class_id) AS number_of_classes
```

```
FROM teacher t
LEFT JOIN teacher_class tc ON t.teacher_id = tc.teacher_id
GROUP BY t.teacher_id, t.last_name, t.first_name
ORDER BY number_of_classes DESC;
```

L.4. Count How Many Teachers Teach Each Class

```
sql
SELECT
   c.class_id,
   c.class_name,
   COUNT(tc.teacher_id) AS number_of_teachers
FROM class c
LEFT JOIN teacher_class tc ON c.class_id = tc.class_id
GROUP BY c.class_id, c.class_name
ORDER BY number_of_teachers DESC;
```

L.5. Find Teachers Who Teach More Than One Class

```
sql
SELECT
   t.teacher_id,
   t.last_name,
   t.first_name,
   COUNT(tc.class_id) AS number_of_classes
FROM teacher t
JOIN teacher_class tc ON t.teacher_id = tc.teacher_id
GROUP BY t.teacher_id, t.last_name, t.first_name
HAVING COUNT(tc.class_id) > 1
ORDER BY number_of_classes DESC;
```

L.6. Find Classes Taught by More Than One Teacher

```
sql
SELECT
   c.class_id,
   c.class_name,
   COUNT(tc.teacher_id) AS number_of_teachers
FROM class c
```

```
JOIN teacher_class tc ON c.class_id = tc.class_id
GROUP BY c.class_id, c.class_name
HAVING COUNT(tc.teacher_id) > 1
ORDER BY number_of_teachers DESC;
```

L.7. Find Teachers Who Teach No Class

```
SELECT
  t.teacher_id,
  t.last_name,
  t.first_name
FROM teacher t
LEFT JOIN teacher_class tc ON t.teacher_id = tc.teacher_id
WHERE tc.class_id IS NULL;
```

L.8. Find Classes with No Teacher

```
SELECT
  c.class_id,
  c.class_name
FROM class c
LEFT JOIN teacher_class tc ON c.class_id = tc.class_id
WHERE tc.teacher_id IS NULL;
```

PART M — Modifying Existing Data with UPDATE

The UPDATE statement modifies existing rows.

General syntax:

```
UPDATE table_name
SET column_name = new_value
WHERE condition;
```

Important: Always use a WHERE clause when you want to modify specific rows. Without WHERE, all rows may be modified.

M.1. Modify One Teacher's Salary

Increase the salary of teacher 101 to 78000:

```
UPDATE teacher
SET salary = 78000
WHERE teacher_id = 101;
```

Verify:

sql

```
SELECT * FROM teacher WHERE teacher_id = 101;
```

M.2. Modify Several Rows

Increase the salaries of teachers in department 1 by 5%:

sql

```
UPDATE teacher
SET salary = salary * 1.05
WHERE department_id = 1
  AND salary IS NOT NULL;
```

Verify:

sql

```
SELECT teacher_id, last_name, salary, department_id
FROM teacher
WHERE department_id = 1;
```

M.3. Modify a Student's Class

Assign student 8 to class 10:

```
UPDATE student
SET class_id = 10
WHERE student_id = 8;
```

Verify:

```
SELECT * FROM student WHERE student_id = 8;
```

M.4. Update Using a Condition

Increase salaries below 50000 by 10%:

```
UPDATE teacher
SET salary = salary * 1.10
WHERE salary < 50000;
```

Verify the affected rows with:

```
SELECT teacher_id, last_name, salary
FROM teacher
WHERE salary < 55000
ORDER BY salary;
```

M.5. Modify a Teacher-Class Assignment

Change teacher 103 from class 30 to class 40:

```
-- Step 1: Delete the old assignment
DELETE FROM teacher_class
WHERE teacher_id = 103 AND class_id = 30;
-- Step 2: Insert the new assignment
INSERT INTO teacher_class (teacher_id, class_id)
VALUES (103, 40);
```

Note: In a junction table, you cannot simply UPDATE a composite key value. You must DELETE the old row and INSERT the new one.

PART N — Basic SELECT Queries

N.1. Select Specific Columns

```
SELECT last_name, first_name FROM teacher;
```

N.2. Use an Alias

sql

```
SELECT last_name, first_name, salary AS monthly_salary
FROM teacher;
```

N.3. Filter with WHERE

Display teachers whose salary is greater than 60000:

sql

```
SELECT last_name, first_name, salary
FROM teacher
WHERE salary > 60000;
```

N.4. Use AND

sql

```
SELECT last_name, first_name, salary
FROM teacher
WHERE department_id = 1
  AND salary > 60000;
```

N.5. Use OR

sql

```
SELECT last_name, first_name, department_id
FROM teacher
WHERE department_id = 1
   OR department_id = 2;
```

N.6. Use BETWEEN

sql

```
SELECT last_name, first_name, salary
FROM teacher
WHERE salary BETWEEN 50000 AND 70000;
```

N.7. Use LIKE

sql

```
SELECT last_name, first_name
FROM teacher
WHERE last_name LIKE 'A%';
```

N.8. Use ILIKE

sql

```
SELECT last_name, first_name
FROM teacher
WHERE last_name ILIKE 'a%';
```

N.9. Find NULL Values

sql

```
SELECT last_name, first_name, salary
FROM teacher
WHERE salary IS NULL;
```

N.10. Find Non-NULL Values

sql

```
SELECT last_name, first_name, salary
FROM teacher
WHERE salary IS NOT NULL;
```

N.11. Sort the Results

sql

```
SELECT last_name, first_name, salary
FROM teacher
ORDER BY salary DESC;
```

PART O — Testing Integrity Constraints

The purpose of these exercises is to observe how PostgreSQL protects data integrity.

O.1. Primary Key Violation

Try:

```
INSERT INTO department (department_id, department_name)
VALUES (1, 'Biology');
```

Questions

1. Why is the insertion rejected?
2. Which constraint is violated?

O.2. Foreign Key Violation

Try:

```
INSERT INTO teacher
  (teacher_id, last_name, first_name, salary, department_id)
VALUES
  (110, 'Test', 'Student', 50000, 99);
```

Question

Why is the insertion rejected?

Hint: department 99 does not exist.

O.3. NOT NULL Violation

Try:

sql

```
INSERT INTO teacher
  (teacher_id, last_name, first_name, salary, department_id)
VALUES
  (111, NULL, 'Test', 50000, 1);
```

Question

Why does PostgreSQL reject this insertion?

O.4. CHECK Violation on student.age

Try:

sql

```
INSERT INTO student
  (student_id, last_name, first_name, age, class_id)
VALUES
  (11, 'Test', 'Student', -5, 10);
```

Question

Which constraint is violated?

O.5. CHECK Violation on teacher.salary

Try:

sql

```
INSERT INTO teacher
   (teacher_id, last_name, first_name, salary, department_id)
VALUES
   (113, 'Test', 'Salary', -1000, 1);
```

Question

Which constraint is violated?

O.6. UNIQUE Violation on class.class_name

Try:

sql

```
INSERT INTO class (class_id, class_name, department_id)
VALUES (60, 'M1-GL', 1);
```

Question

Which constraint is violated?

O.7. CHECK Violation on class.class_name

Try:

sql

```
INSERT INTO class (class_id, class_name, department_id)
VALUES (61, 'AB', 1);
```

Question

Which constraint is violated?

O.8. Composite Primary Key Violation on teacher_class

Try:

sql

```
INSERT INTO teacher_class (teacher_id, class_id)
VALUES (101, 10);
```

Question

Which constraint is violated? Why is this important for the N:N relationship?

O.9. Foreign Key Violation on teacher_class

Try:

sql

INSERT INTO teacher_class (teacher_id, class_id)

VALUES (999, 10);

Question

Which constraint is violated?

O.10. DEFAULT Value Test

Try:

sql

INSERT INTO teacher (teacher_id, last_name, first_name, department_id)

VALUES (112, 'Default', 'Test', 1);

Question

What salary is assigned to this teacher? Why?

PART P — Modification of the Table Structure

The ALTER TABLE statement can also be used to modify the database structure.

P.1. Add a Column

sql

ALTER TABLE student

ADD COLUMN email VARCHAR(100);

Verify:

sql

SELECT * FROM student;

P.2. Drop a Column

If the column is not needed:

sql

ALTER TABLE student

DROP COLUMN email;

P.3. Rename a Column

sql

ALTER TABLE student

RENAME COLUMN age TO student_age;

P.4. Change a Column Type

sql

ALTER TABLE student

ALTER COLUMN student_age TYPE SMALLINT;

P.5. Add a Column to the Junction Table

sql

ALTER TABLE teacher_class

ADD COLUMN academic_year VARCHAR(10);

Update:

sql

UPDATE teacher_class

SET academic_year = '2025-2026';

Verify:

sql

SELECT * FROM teacher_class;

This part introduces structural modification. UPDATE, presented earlier, modifies the data stored in the table.

PART Q — Final Verification

Execute:

sql

SELECT COUNT(*) AS number_of_departments FROM department;

Expected result: **5**

sql

SELECT COUNT(*) AS number_of_teachers FROM teacher;

Expected result: **9**

sql

SELECT COUNT(*) AS number_of_classes FROM class;

Expected result: **5**

sql

SELECT COUNT(*) AS number_of_students FROM student;

Expected result: **10**

sql

SELECT COUNT(*) AS number_of_assignments FROM teacher_class;

Expected result: **13**

Also verify:

- department Chimie exists;
- class M1-Physique exists;
- teacher Khelifi Nora has a NULL salary;
- student Farah Ines has a NULL class_id if the original data state has been restored;
- teacher Benali Ahmed teaches both M1-GL and M1-IA;
- class M1-Math is taught by at least two teachers.

Additional Exercise

PART R — Exercises

Without using a model answer, write SQL queries for the following:

1. Display all department names in alphabetical order.
2. Display the last name, first name, and salary of teachers earning more than 70000.
3. Display teachers whose salary is between 50000 and 80000.
4. Display students younger than 23.
5. Display students whose last name starts with A.
6. Display teachers whose salary is not NULL.
7. Display students who currently have no class.
8. Display all classes ordered by class_name.
9. Display the three highest salaries.
10. Display teachers from department 1 whose salary is greater than 60000.
11. Change the salary of teacher 103 to 52000.
12. Increase all non-NULL salaries in department 3 by 5%.
13. Change the age of student 10 to 21.
14. Explain the difference between INSERT, UPDATE, ALTER TABLE, and SELECT.
15. Add a UNIQUE constraint on class_name and verify it.
16. Modify the chk_student_age constraint to require age >= 18.
17. Drop the UNIQUE constraint on class_name.
18. Display each teacher with the names of the classes they teach.
19. Display each class with the names of the teachers who teach it.
20. Find teachers who teach more than two classes.
21. Find classes taught by exactly one teacher.

22. Find teachers who teach no class.
23. Add a semester column to teacher_class and set it to 'S1' for all rows.
24. Delete a teacher and observe what happens to their rows in teacher_class.
25. Delete a class and observe what happens to its rows in teacher_class.
26. Display all constraints of the teacher table using pg_constraint.
27. Display all constraints of the class table using information_schema.
28. Display the ON DELETE and ON UPDATE rules of all foreign keys in the database.

PART S — Final Challenge

Starting from the university database, perform the following tasks without copying the previous examples:

1. Create a new department named Biology with an appropriate identifier.
2. Create a new teacher belonging to this department.
3. Modify the teacher's salary.
4. Create a new class belonging to the new department.
5. Add a new student to this class.
6. **Assign the new teacher to the new class using** teacher_class**.**
7. **Assign an existing teacher to the new class using** teacher_class**.**
8. Display the inserted rows.
9. Display all teachers of the new class.
10. Display all classes of the new teacher.
11. Explain which primary-key and foreign-key constraints guarantee the consistency of the new data.
12. Explain why the junction table teacher_class does not modify the teacher and class tables.
13. Add a CHECK constraint on the new department name.
14. Drop the CHECK constraint you just added.
15. Add a DEFAULT value on the new teacher's salary.
16. Delete the Biology department and observe what happens to the teacher, class, and teacher_class rows. Explain the result based on the ON DELETE actions defined.